

Miskolci Egyetem

Informatikai Intézet

Objektumorientált programozás

Féléves feladat dokumentáció

JavaFX 3D Teáskanna Nézegető

Hallgató neve: [Hallgató neve]

Neptun kód: [Neptun kód]

Csoport: [Csoport]

Oktató: [Oktató neve]

Tanév: 2024/2025. tanév, tavaszi félév

Leadás dátuma: [Dátum]

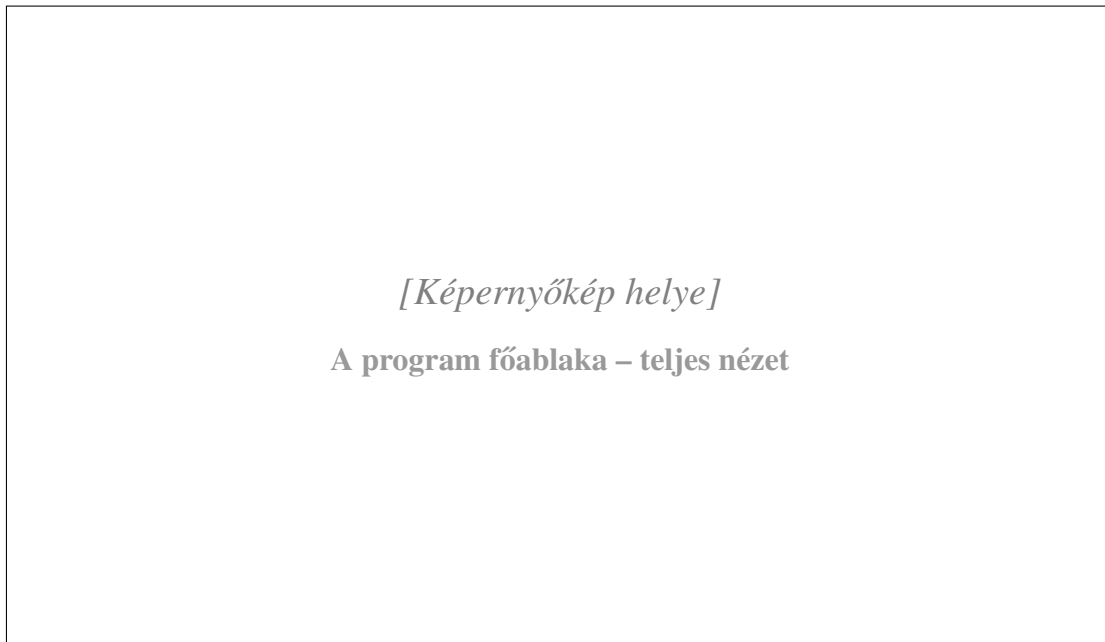
Tartalomjegyzék

1. Bevezetés

1.1. A feladat ismertetése

A feladat célja egy ismert, hétköznapi tárgy háromdimenziós modelljének elkészítése és interaktív megjelenítése JavaFX segítségével. A választott tárgy egy teáskanna, amelynek geometriai felépítése egyszerű forgástestek és összetett hálógenerálási technikák kombinációjával közelíthető meg.

Az alkalmazás lehetővé teszi, hogy a felhasználó a teáskanna összes fontosabb geometriai paraméterét valós időben állítsa be. A módosítások azonnal megjelennek a 3D nézetben: a modell újragenerálódik, és a csúszkák értékeinek megfelelő alakot vesz fel. A programban megtalálható a mentés és betöltés funkció is, így a beállítások JSON fájlba exportálhatók, majd visszatölthetők.



1. ábra. A program főablaka – teljes nézet

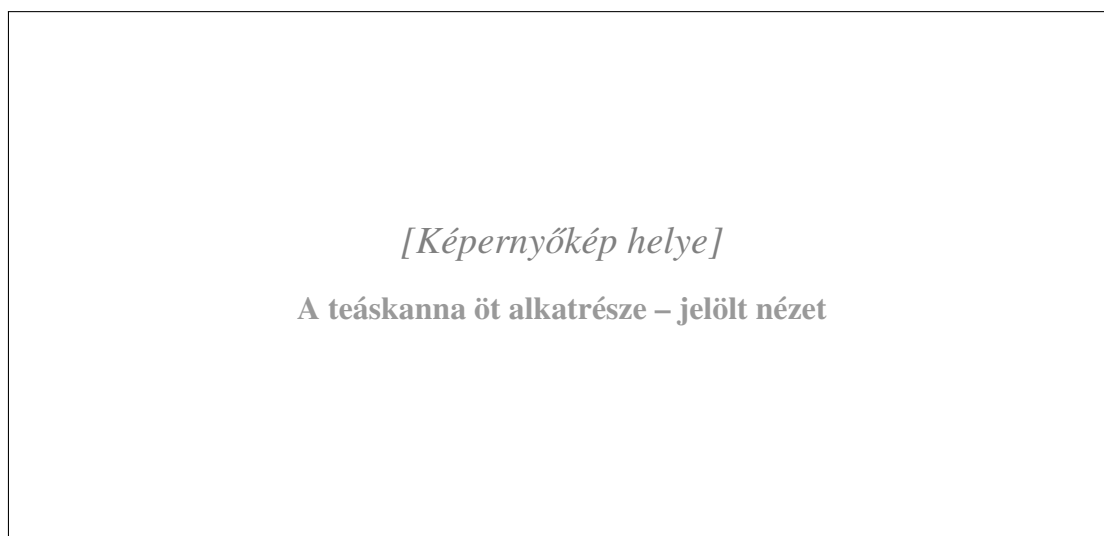
1.2. A program funkciói

Az alkalmazás az alábbi fő funkciókat valósítja meg:

- **Generálás:** Az *Új* menüpont véletlenszerű paraméterekkel hoz létre egy új teáskanna-formát.
- **Törlés / Visszaállítás:** A *Törlés* menüpont az összes paramétert visszaállítja az alapértékekre.
- **Mentés (Exportálás):** Az aktuális paraméterek JSON formátumban fájlba menthetők.

- **Betöltés (Importálás):** Korábban elmentett JSON fájlból visszatölthetők a paraméterek.
- **Rajzolás / Megjelenítés:** A 3D modell valós időben frissül minden paraméterváltozásnál.
- **Bemutatás:** Az egérrel a modell szabadon forgatható, az egérgörgővel nagyítható és kicsinyíthető.

Az objektum öt alkatrészből áll: a váza test, a kiöntő, a fogantyú, a tetőfélgömb és a tető fogógombja. Mindegyik alkatrész saját színnel és paraméterekkel rendelkezik.



2. ábra. A teáskanna öt alkatrésze – jelölt nézet

1.3. Elméleti háttér

A program középpontjában a JavaFX `TriangleMesh` típusa áll. A háromszöghálós reprezentáció a 3D grafika egyik legszélesebb körben használt módszere: az objektum felszínét egymáshoz illeszkedő háromszögek összessége közelíti. Minden háromszöget három csúcspont (vertex) határoz meg, az egymás melletti háromszögek közös csúcsokat és éleket osztanak meg.

A váza körvonalát egy profil-görbe írja le, amelyet forgástestként körbeforgatunk a függőleges tengely körül (surface of revolution). Az egyes körök mentén elhelyezett csúcspontokból a program négyszögeket, majd azokat két-két háromszögre bontva lapokat (face-eket) generál.

A `PhongMaterial` anyagmodell a fény és a felszín kölcsönhatását a diffúz visszaverődés alapján közelíti. Az alkalmazás egy környezeti fényforrást (*ambient light*) és egy pontfényforrást (*point light*) tartalmaz, amelyek egymástól függetlenül ki- és bekapcsolhatók.

Az objektumorientált tervezés szempontjából a program az MVC (Model-View-Controller) mintát követi: a modell tárolja az adatokat, a nézet megjeleníti azokat, a vezérlő pedig összehangolja a kettőt. A JavaFX property-rendszere lehetővé teszi, hogy az adatmodell változásai automatikusan tükröződjenek a felhasználói felületen.

2. Tervezési dokumentáció

2.1. Fejlesztői környezet

A program fejlesztéséhez az alábbi eszközök kerültek felhasználásra:

Eszköz	Verzió	Felhasználás
Java (JDK)	17	Fejlesztési és futtatási platform
JavaFX	21.0.4	3D grafika, kezelőfelület
Maven	3.9	Projektmenedzsment, fordítás, függőségkezelés
IntelliJ IDEA	2024.x	Integrált fejlesztőkörnyezet

1. táblázat. A fejlesztéshez használt eszközök

A `pom.xml` konfigurációs fájl tartalmazza a szükséges függőségeket és a `javafx-maven-plugin` bővítményt, amellyel az alkalmazás közvetlenül futtatható Maven parancsból.

A forráskód a következő főbb JavaFX-technikákat alkalmazza:

- **Property binding:** a csúszkák értéke kötve van a modell tulajdonságaihoz, így minden változás automatikusan kiváltja a háló újragenerálását.
- **TriangleMesh és MeshView:** egyedi háromszöghálók létrehozása és megjelenítése.
- **PhongMaterial:** Phong-árnyalás az egyes alkatrészekre.
- **PerspectiveCamera:** perspektív vetítés a 3D jelenethez.
- **SubScene:** a 3D jelenet és a 2D kezelőfelület elkülönített megjelenítése.

2.2. Csomagstruktúra és osztálydiagram

A projekt Maven-szabványos könyvtárstruktúrát követ, a forrásfájlok a `src/main/java/com/example/view` mappában találhatók.

Listing 1. A projekt csomagstruktúrája

```
com.example.viewer
```

```
+-- MainApp.java
+-- controller/
|   +-- MainController.java
+-- model/
|   +-- VaseParameters.java
+-- view/
|   +-- ControlPanel.java
|   +-- HelpDialog.java
+-- geometry/
|   +-- VaseMeshGenerator.java
|   +-- SpoutMeshGenerator.java
|   +-- HandleMeshGenerator.java
|   +-- LidDomeMeshGenerator.java
|   +-- LidKnobMeshGenerator.java
+-- utils/
    +-- MathUtils.java
    +-- SceneExporter.java
```

Osztályok és felelősségeik:

MainApp az alkalmazás belépési pontja. Kiterjeszti a JavaFX Application osztályt, és a start() metódusban átadja a vezérlést a MainController osztálynak.

VaseParameters tárolja az összes paramétert JavaFX property objektumok formájában. Ezek közvetlenül köthetők csúszkákhoz és egyéb vezérlőkhöz. A handlePos property felülírja a set() metódust, hogy a fogantyú pozíciója mindig a váza magasságán belül maradjon (kényszerfeltétel).

MainController felügyeli a 3D jelenetet, az egérvezérlést és az összes MeshView objektumot. Létrehozza és inicializálja az összes alkatrészt, kezeli a menüpontokat, és összehangolja a fényforrások állapotát.

ControlPanel statikus gyártó osztály. A createControlPanel() metódus az összes csúszkát, színválasztót és jelölőnégyzetet összegyűjti egy görgetőpanelba. A csúszka-értékek megváltozásakor minden alkatrész hálóját újragenerálódik.

HelpDialog egy modális párbeszédablak, amely a program kezelési útmutatóját tartalmazza.

A geometry **csomag** osztályai mind egyetlen statikus metódust tartalmaznak, amely visszaadja az adott alkatrész TriangleMesh objektumát. Ez a kialakítás biztosítja, hogy az egyes geometriagenerátorok egymástól függetlenül fejleszthetők és tesztelhetők legyenek.

MathUtils matematikai segédmetódusokat tartalmaz: a váza profil-sugarának kiszámítását, a smoothStep interpolációs függvényt és a kiöntő súlymaszk-számítását.

SceneExporter az exportálási és importálási logikát valósítja meg. Külső könyvtár nélkül,

kézzel írott JSON-sorosítással dolgozik.



[Képernyőkép helye]
UML osztálydiagram

3. ábra. UML osztálydiagram

2.3. Algoritmusok leírása

2.3.1. Váza profil és forgástest-generálás

A váza külső sugárát az alábbi képlet adja meg minden t paraméterértékre (ahol $t \in [0, 1]$ a relatív magasság):

$$r(t) = r_0 + b \cdot \sin(\pi t) + 9 \cdot \sin(2,3\pi t + 0,4) - n \cdot t^{1,4} \quad (1)$$

ahol r_0 az alapsugár (baseRadius), b a hasasodás mértéke (bellyAmount), n a nyakszűkület (neckTaper).

A $\sin(\pi t)$ tag biztosítja a hasasodást: az alján és tetején kisebb, középen nagyobb a sugár. A második szinuszos tag finomabb hullámos felületet ad. A $n \cdot t^{1,4}$ kifejezés a felső rész fokozatos szűkülését modellezi.

A forgástest-generálás lépései:

1. A program 64 vízszintes gyűrűt számol ki a magasság mentén.

2. Minden gyűrűn 72 (alapértelmezés szerint) pontot helyez el egyenlő szögtávolságra.
3. A szomszédos gyűrűk pontjait két háromszöggel köti össze (négyszögfelosztás).
4. Az alsó laphoz középpontot vesz fel, amelyből háromszögek mutatnak kifelé.
5. A belső falat ugyanígy generálja, de a sugárból kivonja a falvastagságot; a felső és alsó perem összekötésével zárt testet kap.

2.3.2. Kiöntő generálása

A kiöntő nem külön testként, hanem a váza testének deformációjaként jelenik meg. Minden csúcspont esetén kiszámít egy `spoutWeight` értéket ($\in [0, 1]$), amely megmondja, hogy az adott pont mennyire vesz részt a kiöntőben.

Listing 2. A kiöntő súlymaszk-számítása (MathUtils.java)

```

1 float topMask      = smoothStep(0.68f, 1.0f, t);           // csak a
   felso 32%
2 float widthRad     = toRadians(spoutWidth * 0.5);         //
   szogszelesség
3 float angularMask  = exp(-(centeredAngle / widthRad)^2);  // Gauss
   -ablak
4 spoutWeight        = topMask * angularMask;

```

A `smoothStep` függvény: $s(t) = t^2(3 - 2t)$, amely derivált-folytonos átmenetet biztosít. A kiöntő csúcspontja eltolódik kifelé ($r += \text{spoutLength} \cdot w$) és felfelé ($y += \text{spoutLift} \cdot w$). Ezáltal a kiöntő sima átmenettel olvad bele a váza testébe.

2.3.3. Fogantyú generálása

A fogantyú egy félkör ívből kihúzott tóruszcikk. Az ív 33 pont mentén halad (32 szegmens), minden pontban egy 9 csúcsból álló körzetes keresztmetszettel. A háló 32×8 négyszöglapot tartalmaz.

A fogantyú pozíciójának korlátja automatikusan érvényesül: a `VaseParameters.handlePos` property `set()` módszere mindig a váza magasságán belülre szorítja az értéket, a `ControlPanel.enforceH` módszer pedig a fogantyú méretét is korlátozza, hogy ne lógjon ki a teáskanna keretéből.

2.3.4. Exportálás és importálás

Az exportálás kézzel írott JSON formátumot alkalmaz, külső könyvtár nélkül. Az importálás reguláris kifejezésekkel elemzi a fájlt, és ha valamely kulcs hiányzik, az aktuális értéket tartja

meg (graceful degradation). A fájlkezelés szabványos Java I/O osztályokkal (`FileWriter`, `BufferedReader`) történik.

3. Felhasználói dokumentáció

3.1. Telepítés

Az alkalmazás futtatásához az alábbiak szükségesek:

- **Java Development Kit (JDK) 17 vagy újabb.** Letölthető az `adoptium.net` oldalról. Telepítés után a `java -version` parancsnak legalább `17.x.x` verziószámot kell mutatnia.
- **Apache Maven 3.6 vagy újabb.** Letölthető a `maven.apache.org` oldalról. A `mvn -version` paranccsal ellenőrizhető.

A JavaFX könyvtárakat a Maven automatikusan letölti az első fordítás során, külön telepítés nem szükséges. Az alkalmazás forráskódját tartalmazza a mellékelt ZIP archívum, amelyet tetszőleges mappába kell kicsomagolni.

3.2. Futtatás

A program terminálból (Windows: Command Prompt vagy PowerShell) indítható el. Először a projekt mappájába kell navigálni:

```
cd C:\[el r s i t ]\javafx-3d-object-viewer
```

Ezt követően a fordítás és futtatás egyetlen paranccsal elvégezhető:

```
mvn javafx:run
```

Az első futtatásnál a Maven letölti a szükséges függőségeket, ez néhány percet vehet igénybe aktív internetkapcsolat esetén. Ha csak fordítani szeretnénk a futtatás nélkül:

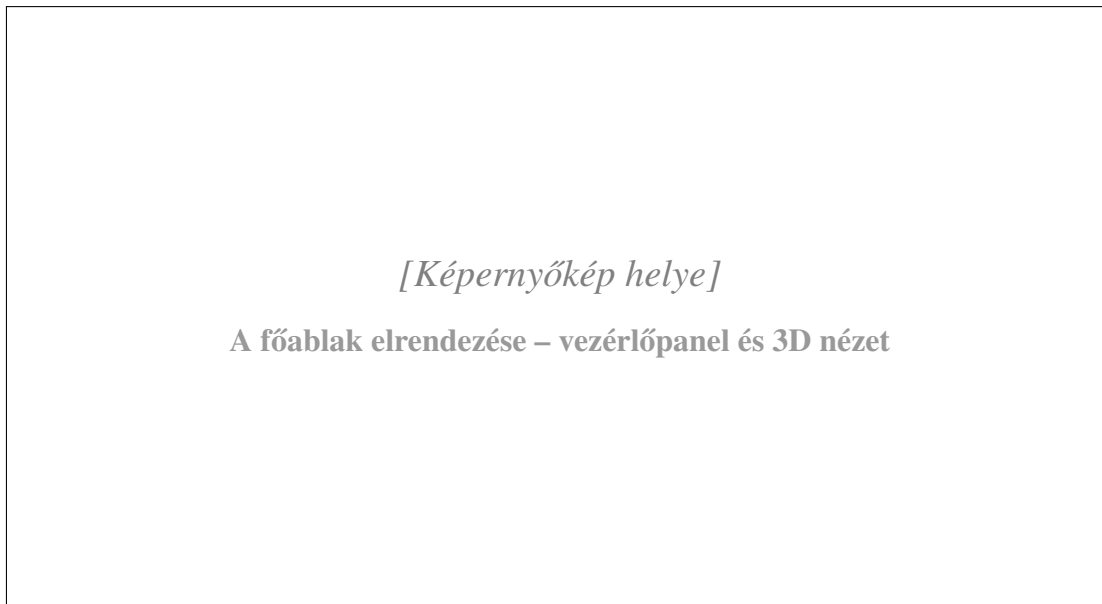
```
mvn compile
```

3.3. Menüstruktúra és kezelőfelület

Főablak

Az alkalmazás ablaka két fő részre oszlik:

- **Bal oldali vezérlőpanel:** itt találhatók a csúszkák, a színválasztók és a fény-kapcsolók. A panel görgethető, ha a tartalom nem fér el.
- **Jobb oldali 3D nézet:** a teáskanna interaktívan forgatható és nagyítható.

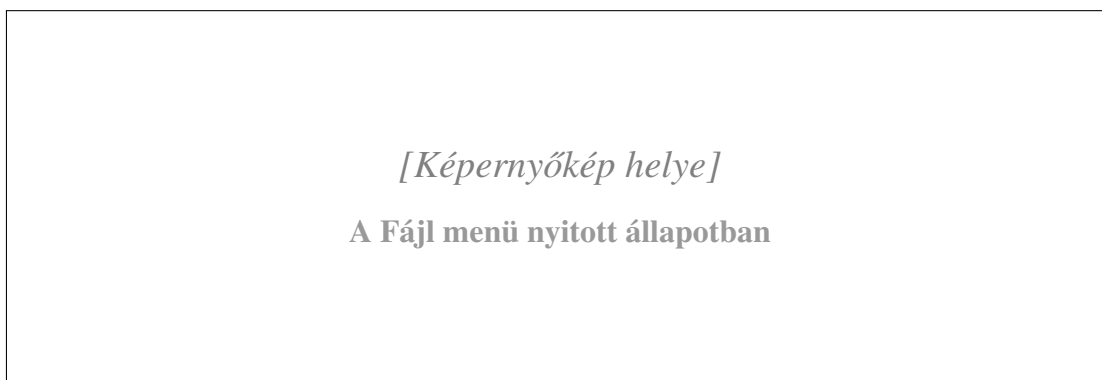


4. ábra. A főablak elrendezése – vezérlőpanel és 3D nézet

Fájl menü

Menüpont	Funkció
Új	Véletlenszerű paraméterekkel generál egy új kannaformát
Törlés	Az összes paramétert visszaállítja az alapértékekre
Exportálás...	Fájlmentési párbeszédet nyit; a paramétereket JSON fájlba menti
Importálás...	Fájlmegnyitási párbeszédet nyit; JSON fájlból tölti be a paramétereket

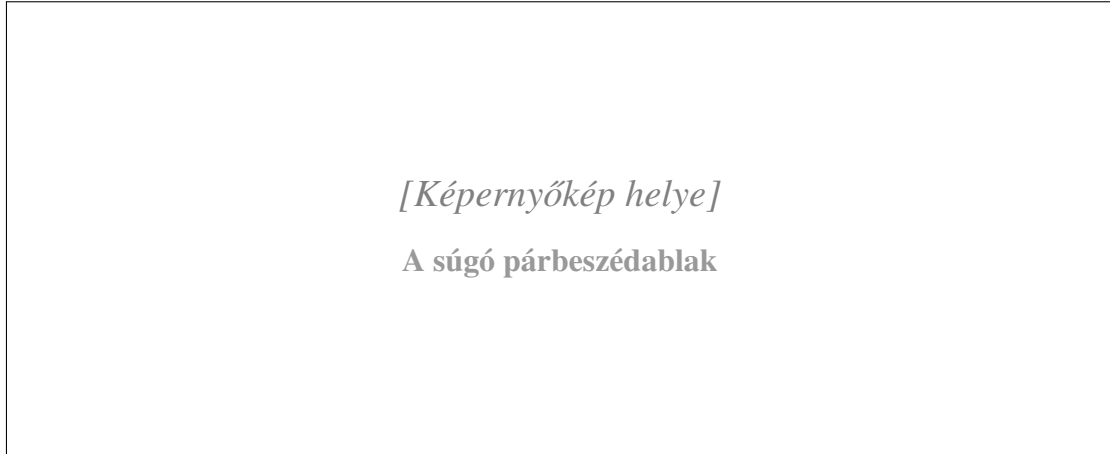
2. táblázat. Fájl menü elemei



5. ábra. A Fájl menü nyitott állapotban

Súgó menü

A *Súgó megjelenítése* menüpont egy modális ablakot nyit meg, amely összefoglalja az egérvezérlés, a csúszkák, a fájlkezelés és az alkatrészek kezelésének módját.

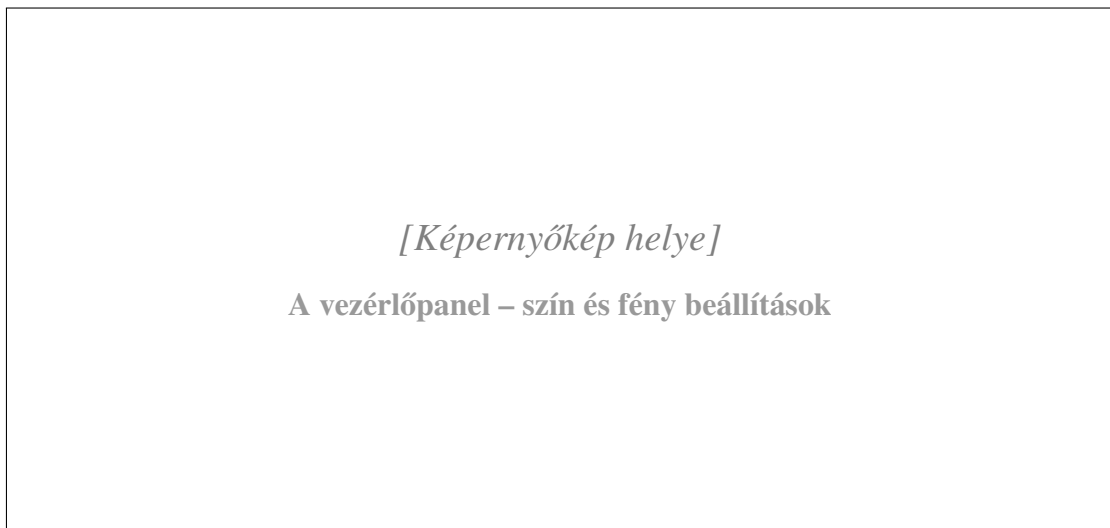


6. ábra. A súgó párbeszédablak

Vezérlőpanel részletei

Szín beállítások: Négy ColorPicker vezérlő áll rendelkezésre a test, a fogantyú, a tető félgömb és a tető fogógombja számára. A szín megváltoztatása azonnal megjelenik a 3D nézetben.

Fények: Két jelölőnégyzet (CheckBox) kapcsolja be és ki a környezeti fényt és a pontfényforrást.



7. ábra. A vezérlőpanel – szín és fény beállítások

Váza paraméterek:

Csúszka	Tartomány	Leírás
Magasság	170 – 280	A váza teljes magassága
Falvastagság	4 – 10	A váza falának vastagsága
Hasasodás	6 – 46	Mennyire pocakos a váza középső része
Nyakszűkület	0 – 22	Mennyire szűkül a váza fölfelé haladva
Alsó sugár	30 – 62	A váza alapsugárának mérete
Felbontás	24 – 128	A radiális szegmensek száma; nagyobb érték simább felületet ad

3. táblázat. Váza alapparaméterek

Kiöntő paraméterek:

Csúszka	Tartomány	Leírás
Kiöntő hossza	4 – 30	Mennyire áll ki a kiöntő
Kiöntő szélessége	15 – 120	A kiöntő szélessége fokban
Ajak emelés	–20 – 18	A kiöntő ajkának függőleges eltolása

4. táblázat. Kiöntő paraméterek

Tető paraméterek:

Csúszka	Tartomány	Leírás
Tető magasság	20 – 56	A tetőfélgömb magassága
Fogó magasság	8 – 42	A tető fogógombjának magassága
Fogó sugár	10 – 20	A fogógomb alapsugárának mérete

5. táblázat. Tető paraméterek

Fogantyú paraméterek:

Csúszka	Tartomány	Leírás
Fogó méret	20 – 80	A fogantyú ívsugárának mérete
Fogó pozíció	–140 – 140	A fogantyú függőleges helyzete a vázán
Fogó vastagság	4 – 16	A fogantyú keresztmetszetének átmérője

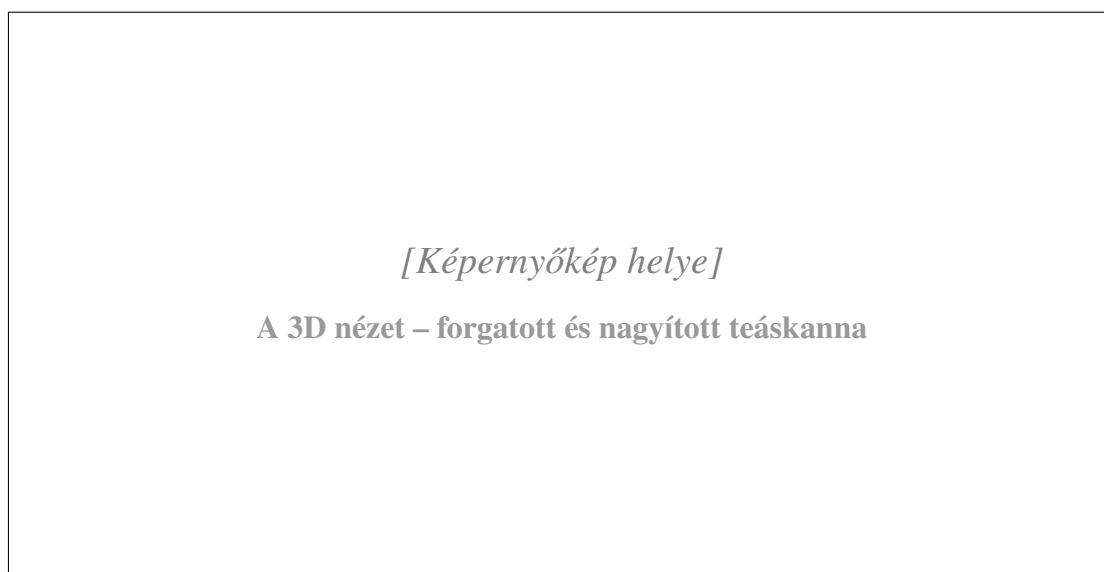
6. táblázat. Fogantyú paraméterek

3D nézet vezérlése

Mozdulat	Hatás
Bal egérgomb + húzás	A modell forgatása X és Y tengely körül
Egérgörgő felfelé	Közelítés (zoom in)
Egérgörgő lefelé	Távolítás (zoom out)

7. táblázat. 3D nézet egérvezérlése

A kamera távolságának határa 200 és 2000 egység között van rögzítve, hogy a modell ne tűnhesen el a látótérből.



8. ábra. A 3D nézet – forgatott és nagyított teáskanna

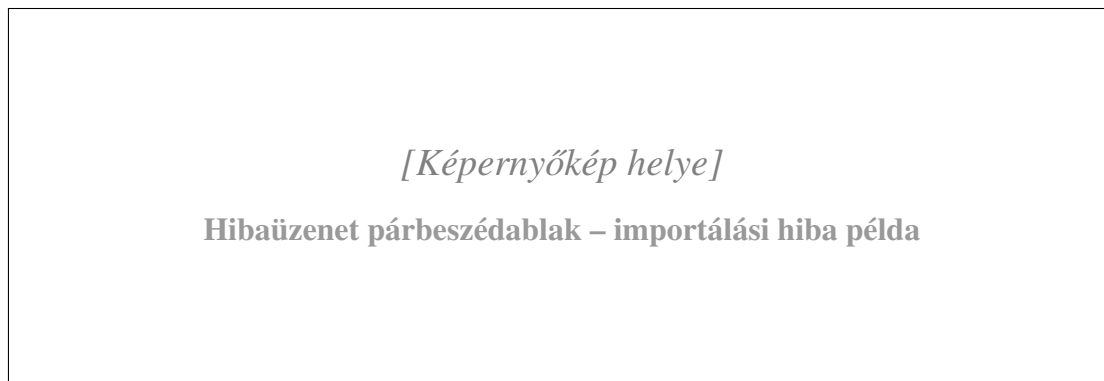
3.4. Hibakezelés

Importálási hiba: Ha a kiválasztott JSON fájl olvasása közben hiba lép fel (pl. sérült fájl, hiányzó kulcsok), az alkalmazás hibaüzenetes párbeszédablakot jelenít meg, és megtartja az aktuálisan érvényes paramétereket. Részlegesen érvényes fájlok esetén az olvasható értékek betöltődnek, a hiányzók helyett az előző értékek maradnak.

Exportálási hiba: Ha az exportálás során írási hiba lép fel (pl. nincs írási jog a kiválasztott mappában), a program szintén hibaüzenetet jelenít meg.

Fogantyú korlátok: A fogantyú pozíciójára és méretére automatikus kényszerfeltétel vonatkozik. Ha a magasság csúszkájának megváltoztatása után a fogantyú kilógna a vázáról, a program automatikusan a megengedett tartományba korrigálja az értéket. Erről értesítés nem jelenik meg, mivel a korrekció folyamatos és a felhasználó számára láthatatlan marad.

Ismeretlen JavaFX hiba: Ha a Java vagy a JavaFX nincs megfelelően telepítve, a program terminálban hibaüzenetet ír ki. Ilyen esetben ellenőrizni kell a JDK és a Maven verzióját.



9. ábra. Hibaüzenet párbeszédablak – importálási hiba példa

4. Irodalomjegyzék

1. Sharan, K.: *Learn JavaFX 17 – Building User Experience and Interfaces with Java*. Apress, 2022.
2. Bloch, J.: *Effective Java*. 3. kiadás. Addison-Wesley, 2018.
3. Oracle Corporation: *JavaFX API Documentation*. [online] Elérhető: <https://openjfx.io/javadoc/21/> [Letöltve: 2025. január]
4. OpenJFX Contributors: *OpenJFX Getting Started Guide*. [online] Elérhető: <https://openjfx.io/openjfx-docs/> [Letöltve: 2025. január]
5. Apache Software Foundation: *Maven Getting Started Guide*. [online] Elérhető: <https://maven.apache.org/guides/getting-started/> [Letöltve: 2025. február]
6. Hughes, J. F., Van Dam, A., McGuire, M. et al.: *Computer Graphics: Principles and Practice*. 3. kiadás. Addison-Wesley, 2014.

5. Melléklet

5.1. Forráskód

A forráskód a projekt `src/main/java/com/example/viewer/` könyvtárában található. A következő osztályok teljes kódja kerül közzétre; a geometriageneráló osztályok (geometry csomag) a mellékelt ZIP archívumban olvashatók.

MainApp.java

Listing 3. MainApp.java – belépési pont

```

1 package com.example.viewer;
2
3 import com.example.viewer.controller.MainController;
4 import javafx.application.Application;
5 import javafx.stage.Stage;
6
7 public class MainApp extends Application {
8
9     private MainController controller;
10
11     @Override
12     public void start(Stage stage) {
13         controller = new MainController();
14         controller.start(stage);
15     }
16
17     public static void main(String[] args) {
18         launch(args);
19     }
20 }

```

model/VaseParameters.java

Listing 4. VaseParameters.java – adatmodell

```

1 package com.example.viewer.model;
2
3 import javafx.beans.property.DoubleProperty;
4 import javafx.beans.property.IntegerProperty;
5 import javafx.beans.property.SimpleDoubleProperty;
6 import javafx.beans.property.SimpleIntegerProperty;
7 import javafx.scene.paint.Color;
8
9 public class VaseParameters {
10     public final DoubleProperty height = new
11         SimpleDoubleProperty(260);
12     public final DoubleProperty wallThickness = new
13         SimpleDoubleProperty(10);
14     public final DoubleProperty bellyAmount = new
15         SimpleDoubleProperty(22);

```

```
13     public final DoubleProperty neckTaper          = new
        SimpleDoubleProperty(4);
14     public final DoubleProperty baseRadius         = new
        SimpleDoubleProperty(38);
15     public final IntegerProperty radialSegments    = new
        SimpleIntegerProperty(72);
16
17     public final DoubleProperty spoutLength        = new
        SimpleDoubleProperty(12);
18     public final DoubleProperty spoutWidth         = new
        SimpleDoubleProperty(70);
19     public final DoubleProperty spoutLift          = new
        SimpleDoubleProperty(7);
20
21     public final DoubleProperty lidHeight          = new
        SimpleDoubleProperty(30);
22     public final DoubleProperty knobHeight         = new
        SimpleDoubleProperty(20);
23     public final DoubleProperty knobRadius         = new
        SimpleDoubleProperty(12);
24
25     public final DoubleProperty handleSize         = new
        SimpleDoubleProperty(40);
26     public final DoubleProperty handleThickness    = new
        SimpleDoubleProperty(8);
27
28     public final DoubleProperty handlePos = new
        SimpleDoubleProperty(0) {
29         @Override
30         public void set(double value) {
31             double min = -height.get() / 2;
32             double max =  height.get() / 2;
33             super.set(Math.max(min, Math.min(max, value)));
34         }
35     };
36
37     public final javafx.beans.property.ObjectProperty<Color>
        bodyColor =
38         new javafx.beans.property.SimpleObjectProperty<>(Color.
            rgb(210, 130, 80));
39     public final javafx.beans.property.ObjectProperty<Color>
        handleColor =
```



```

40     new javafx.beans.property.SimpleObjectProperty<>(Color.
        rgb(180, 100, 60));
41 public final javafx.beans.property.ObjectProperty<Color>
    lidDomeColor =
42     new javafx.beans.property.SimpleObjectProperty<>(Color.
        rgb(196, 116, 72));
43 public final javafx.beans.property.ObjectProperty<Color>
    lidKnobColor =
44     new javafx.beans.property.SimpleObjectProperty<>(Color.
        rgb(220, 140, 80));
45
46 public final javafx.beans.property.BooleanProperty
    ambientLightEnabled =
47     new javafx.beans.property.SimpleBooleanProperty(true);
48 public final javafx.beans.property.BooleanProperty
    pointLightEnabled =
49     new javafx.beans.property.SimpleBooleanProperty(true);
50 }

```

utils/MathUtils.java

Listing 5. MathUtils.java – matematikai segédfüggvények

```

1 package com.example.viewer.utils;
2
3 public class MathUtils {
4
5     public static float smoothStep(float edge0, float edge1,
        float x) {
6         float t = (x - edge0) / (edge1 - edge0);
7         t = Math.max(0f, Math.min(1f, t));
8         return t * t * (3f - 2f * t);
9     }
10
11     public static float findMaxVaseRadius(
        com.example.viewer.model.VaseParameters params) {
12         float maxRadius = 0f;
13         for (int i = 0; i <= 120; i++) {
14             float t = (float) i / 120f;
15             maxRadius = Math.max(maxRadius, vaseProfileRadius(t,
                params));
16         }
17         return maxRadius;
18     }

```

```

19     }
20
21     public static float vaseProfileRadius(
22         float t, com.example.viewer.model.VaseParameters
23         params) {
24         return (float) params.baseRadius.get()
25             + (float) params.bellyAmount.get() * (float) Math.
26               sin(Math.PI * t)
27             + 9f * (float) Math.sin(2.3 * Math.PI * t + 0.4)
28             - (float) params.neckTaper.get() * (float) Math.pow
29               (t, 1.4);
30     }
31
32     public static float computeSpoutWeight(
33         float t, double angle,
34         com.example.viewer.model.VaseParameters params) {
35         float topMask = smoothStep(0.68f, 1.0f, t);
36         float widthRad = (float) Math.toRadians(params.
37             spoutWidth.get() * 0.5);
38         widthRad = Math.max(0.12f, widthRad);
39         float centered = (float) Math.atan2(
40             Math.sin(angle - Math.PI), Math.cos(angle - Math.PI)
41             );
42         float angularMask = (float) Math.exp(
43             -Math.pow(centered / widthRad, 2.0));
44         return topMask * angularMask;
45     }
46 }

```

5.2. Értékelő lap

Szempont	Max. pont	Kapott pont
Dokumentáció	5	
Program működés	5	
Megvalósítás minősége	5	
Bemutató	5	
Összesen	20	

8. táblázat. Értékelő lap

Pont	Jegy
0 – 7	1 (elégtelen)
8 – 10	2 (elégséges)
11 – 13	3 (közepes)
14 – 16	4 (jó)
17 – 20	5 (jeles)

9. táblázat. Ponthatárok és jegyek

Elért jegy: _____

Megjegyzés: _____

Aláírás: _____